Moreover, nodes will hold pointers (or references) to two children, left and right, and their parent.

The constructor for a node will just set the key and priority attributes from its arguments, and initialize left and right pointers to `null`, effectively creating a leaf; the two branches can then be set after construction (or, alternatively, an overloaded version of the constructor also taking the two children can be provided).

3.3.1 Rotations

How do we get out of the impasse? There is one operation on binary search trees that can help us: rotations. Figure 3.4 illustrates how rotations can heal (or break!) the constraints on a treap; rotations are common operations on many versions of BSTs, like red-black trees or 2-3 trees[3].

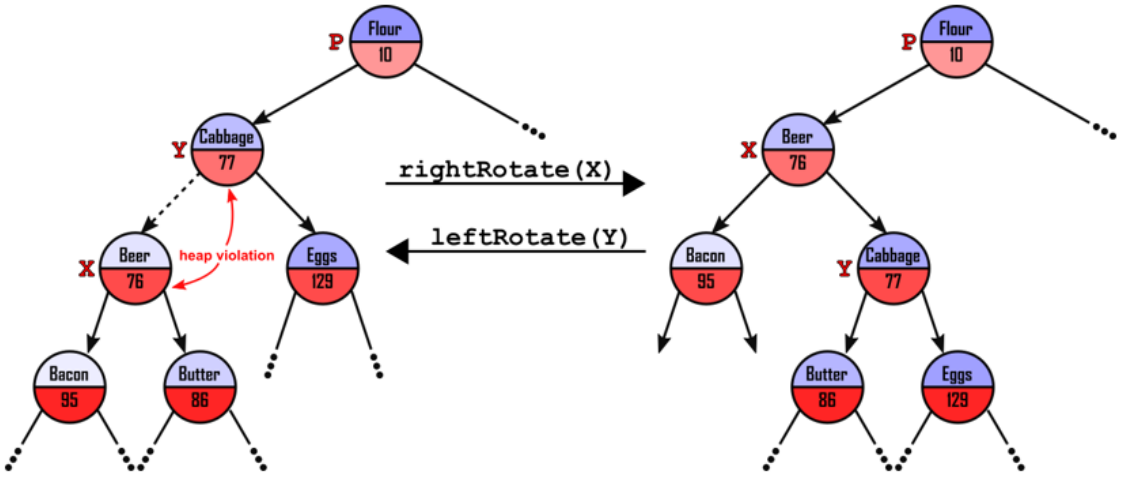


Figure 3.4 Left and right rotations illustrated on an example. The treap on the left violates heap's invariants, and a right rotation on the node marked as `X` can fix it. Conversely, if we apply a left rotation to the node marked with `Y` in the heap on the right, the result will break heap's constraints.